实验报告：丢包检测机制实现

王艺杭
2023202316

1 实验任务与目标

1.1 任务背景

在之前的实验中，发送端和接收端主要工作在理想链路条件下，发送端发出的数据报文能够被接收端顺利接收并确认。但在真实网络中，报文丢失和乱序是不可避免的。一旦某个数据报文丢失，接收端为了保持按序交付，就不能继续推进确认点，而发送端也需要尽快感知这一异常，以便后续触发对应的拥塞控制或重传策略。

本次实验聚焦于“丢包检测”本身，不要求实现丢包后的恢复，只要求在当前框架中补全两种最基本的检测信号：

1. 发送端长时间收不到新的确认时，通过超时发现异常
2. 发送端收到多个确认号相同的 ACK 时，通过重复 ACK 发现异常

1.2 任务目标

1. 修改 `receiver.cc`，恢复“每收到一个新数据报文就发送一个 ACK”的基本语义
2. 在接收端实现重复 ACK：发生乱序时继续反馈最后一个按序收到的数据报文序号
3. 在 `controller.cc` 中实现超时检测后的窗口减半，并输出 `timeout`
4. 在 `controller.cc` 中检测重复 ACK，并输出 `got same ack <seq>`
5. 编写实验说明，说明算法原理、实现细节和验证方法

1.3 技术要求

- 不修改现有协议头格式，不改 `contest_message.cc` 的序列化与反序列化逻辑
- 不实现 RACK，不实现丢包重传，不实现乱序缓存
- 保持 `Controller` 的公开接口不变，不改 `sender.cc` 的调用方式

---

2 实验原理与机制说明

2.1 超时检测原理

超时检测是最基础的丢包检测方法。发送端发出数据后，会等待接收端返回 ACK。如果在设定的超时时间内没有收到新的确认，就认为当前链路存在较大概率的异常，可能是报文丢失，也可能是网络拥塞或严重乱序。

本实验中，发送端已经在 `sender.cc` 的主循环中集成了超时触发逻辑：如果 `poll()` 超时，就会调用：

```
send_datagram(true);
```

因此控制器只需要在 `datagram_was_sent(..., after_timeout)` 中识别 `after_timeout == true` 的情况，即可认定发生了超时。

2.2 重复 ACK 原理

接收端向上层交付数据时必须保证有序，因此如果某个序号的数据报文丢失了，即使后续更大的序号已经到达，接收端也不能把确认点继续向前推进，而是要持续反馈“最后一个按序收到的序号”。这样发送端就会连续收到多个确认号相同的 ACK，这些 ACK 就是重复 ACK。

例如，接收端已经按序收到了 `0, 1, 2`，因此当前期望收到的下一个报文是 `3`。如果报文 `4` 先到达，说明链路上可能发生了丢包或乱序。此时接收端仍应返回 ACK `2`，也就是最后一个按序收到的报文序号。若后续 `5` 也先到达，则仍继续返回 ACK `2`，从而在发送端形成连续重复 ACK。

2.3 为什么要回退 task2 的延迟 ACK

开发任务二实现的是“每收到两个数据报文反馈一个 ACK”的延迟 ACK 机制，该逻辑会改变 ACK 的触发节奏。开发任务三则需要观察“丢包时 ACK 不前进”的现象，如果保留 2:1 延迟 ACK，ACK 的数量和时机会被额外改变，不利于直接判断某个 ACK 是因为乱序触发的重复 ACK，还是因为延迟 ACK 机制本身导致的反馈减少。

因此本次实验应先恢复最基础的逐包 ACK 语义，再在其上实现重复 ACK。

---

3 实验过程与实现细节

3.1 修改 `controller.hh`

为了让控制器能记住当前窗口大小和最近一次收到的 ACK，在 `Controller` 类中新增了三个私有成员：

```
unsigned int window_size_;
uint64_t last_ack_;
bool has_last_ack_;
```

它们的作用分别是：

- `window_size_`：保存当前发送窗口大小，初始值为 50
- `last_ack_`：保存上一条 ACK 的确认号
- `has_last_ack_`：标记是否已经收到过第一条 ACK，避免程序启动初期误判

3.2 修改 `controller.cc`

3.2.1 1. 动态返回窗口大小

将 `window_size()` 从固定返回 50 改为返回成员变量 `window_size_`：

```
unsigned int Controller::window_size() {
  return window_size_;
}
```

3.2.2 2. 超时后打印并减窗

在 `datagram_was_sent()` 中判断 `after_timeout`：

```
if (after_timeout) {
  cout << "timeout" << endl;
  window_size_ = max(1u, window_size_ / 2);
}
```

这样每次超时都会输出 `timeout`，并将窗口减半，同时通过 `max(1u, ...)` 保证窗口不会下降到 0。

3.2.3 3. 检测重复 ACK

在 `ack_received()` 中保留原有的 ACK 打印，同时比较当前 ACK 和上一条 ACK 是否相同：

```
cout << "num_acked:" << sequence_number_acked << endl;

if (has_last_ack_ && sequence_number_acked == last_ack_) {
  cout << "got same ack " << sequence_number_acked << endl;
}

last_ack_ = sequence_number_acked;
has_last_ack_ = true;
```

如果 ACK 序号未发生变化，就说明接收端再次确认了同一个“最后按序收到的报文”，此时即可判定出现了重复 ACK。

3.3 修改 `receiver.cc`

本次实验不再使用 task2 中的 `packet_count`、`ACK_INTERVAL` 等延迟 ACK 状态，而是改为维护两个新的变量：

```
uint64_t ack_sequence_number = 0;
uint64_t next_expected_data_sequence = 0;
```

- `ack_sequence_number`：ACK 报文自身的序列号，单调递增
- `next_expected_data_sequence`：当前期望收到的下一个数据报文序号

核心逻辑如下：

```
const uint64_t received_sequence = message.header.sequence_number;
if (received_sequence == next_expected_data_sequence) {
  next_expected_data_sequence++;
} else if (received_sequence > next_expected_data_sequence and
           next_expected_data_sequence > 0) {
  message.header.sequence_number = next_expected_data_sequence - 1;
}

message.transform_into_ack(ack_sequence_number++, recd.timestamp);
message.set_send_timestamp();
socket.sendto(recd.source_address, message.to_string());
```

这段逻辑对应两种情况：

1. **按序到达**
   当前收到的数据报文正好是接收端期望的那个序号，此时确认当前报文，并把期望序号推进一位。
2. **乱序到达**
   当前收到的序号比期望值更大，说明中间至少缺了一个包。此时先把 `message.header.sequence_number` 手动改写成 `next_expected_data_sequence - 1`，再调用 `transform_into_ack()`。由于 `transform_into_ack()` 会把原来的 `header.sequence_number` 写入 `header.ack_sequence_number`，所以改写后发出的就是重复 ACK。

3.4 实现边界说明

本实验没有实现乱序缓存，因此后到达的更大序号数据报文不会被保留下来等待补洞，只用于触发重复 ACK。

另外，当 `0` 号数据报文一开始就丢失时，协议头中没有“尚未按序确认任何数据报文”的专门表示，因此本实现默认按照实验文档中的典型场景处理，即链路已经建立了正常的按序接收进度后，再出现丢包或乱序。

4 实验验证与结果分析

4.1 1. 构建验证

在仓库根目录执行：

```
make
```

代码可以正常通过编译，说明本次修改没有破坏现有接口与构建流程。

4.2 2. 无丢包场景验证

在无丢包环境下运行 sender 和 receiver，`controller.cc` 会持续打印：

```
num_acked:0
num_acked:1
num_acked:2
num_acked:3
...
```

此时 ACK 序号应基本单调递增，并且不会出现 `got same ack`，说明逐包 ACK 逻辑正常工作。

4.3 3. 超时场景验证

如果链路长时间没有返回新的 ACK，发送端会调用 `send_datagram(true)`，控制器对应打印：

```
timeout
```

并把窗口按如下方式缩小：

```
50 -> 25 -> 12 -> 6 -> 3 -> 1
```

由于使用了 `max(1u, window_size_ / 2)`，窗口最小保持为 1，不会降为 0。

4.4 4. 重复 ACK 场景验证

仓库提供的 `datagrump/run-contest` 会通过 Mahimahi 构造链路环境。为了观察重复 ACK，可以临时把命令链中的：

```
my @command = qw{mm-delay 20 mm-link UPLINK DOWNLINK};
```

改成：

```
my @command = qw{mm-delay 20 mm-loss uplink 0.01 mm-link UPLINK DOWNLINK};
```

随后重新运行 sender 和 receiver。此时若出现某个数据报文在 uplink 上丢失，而后续报文继续到达，则发送端可能观察到类似输出：

```
num_acked:18
num_acked:18
got same ack 18
num_acked:18
got same ack 18
num_acked:19
```

这说明：

- 接收端已经把确认点卡在 `18`

- 后续更大的序号虽然到达，但因为前面有缺口，所以继续返回 ACK `18`
- 当缺失的报文补齐或新的按序确认建立后，ACK 才会继续向前推进

4.5 结果总结

从验证结果可以看出，本实验成功实现了两类典型的丢包检测信号：

1. **超时检测**：ACK 长时间不前进时，控制器能够输出 `timeout` 并缩小窗口
2. **重复 ACK 检测**：当接收端遇到乱序报文时，能够持续反馈相同的 ACK，发送端识别后输出 `got same ack <seq>`

虽然本实验并未实现基于这些信号的重传或更复杂的拥塞控制，但已经为后续扩展可靠传输协议打下了基础。

---

5 实验总结

本次实验围绕“如何发现丢包”完成了接收端和控制器两部分改造。相比前一次实验的延迟 ACK，本次实现回到了更基础的逐包确认语义，使得 ACK 的推进与停滞能够更直接地反映链路状态。

从工程实现角度看，这个实验的关键不在于增加复杂算法，而在于理清几个序列号之间的职责：

- 数据报文的 `sequence_number` 由发送端维护
- ACK 报文头中的 `ack_sequence_number` 表示接收端当前确认到的数据报文序号
- ACK 报文自己的 `sequence_number` 只是接收端为 ACK 分配的编号，与丢包检测本身无关

通过这次实现，可以更清晰地理解可靠传输协议中的一个核心思想：发送端并不一定要“看到丢失本身”，只要通过 ACK 的停滞、重复和超时这些外部现象，就能推断出链路上很可能发生了异常。